



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 14

“Vúmetro”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 15 de abril de 2025

INTRODUCCIÓN

Vúmetro

Un vúmetro (del inglés Volume Unit Meter) es un instrumento de medición que indica el nivel de señal de audio en un equipo de sonido. A diferencia de un vólmetro convencional, el vúmetro está diseñado para representar la sonoridad percibida por el oído humano, lo que lo convierte en una herramienta esencial en la ingeniería de audio y la radiodifusión.

Naturaleza de la Medición (La Unidad VU)

El vúmetro mide el valor promedio de la señal de audio. Se calibra para que el **0 VU** represente el nivel óptimo de grabación o transmisión, evitando la distorsión.

- **Escala Logarítmica:** Dado que el oído humano percibe el sonido de forma logarítmica (decibelios), la escala del vúmetro no es lineal. El rango suele ir de -20 a +3 VU.
- **Tiempo de Respuesta (Balística):** Un vúmetro estándar tiene un tiempo de respuesta de aproximadamente 300 milisegundos. Esto significa que la aguja o los LEDs no reaccionan a picos instantáneos, sino al promedio de la potencia, imitando cómo el cerebro procesa la intensidad sonora.

Tipos de Vúmetros

Vúmetros Analógicos (De Aguja)

Utilizan un galvanómetro de bobina móvil. Son muy apreciados en el audio profesional "vintage" por su suavidad de movimiento.

- **Rectificación:** Como el audio es una señal de CA (Corriente Alterna), el vúmetro analógico incluye un puente rectificador interno para que la aguja se mueva siempre hacia la derecha.

Vúmetros de LEDs (Barras de Luz)

Son los más comunes en la actualidad. Utilizan una serie de comparadores de voltaje para encender una fila de LEDs.

- **Código de Colores:** Generalmente son Verdes (nivel seguro), Amarillos (advertencia) y Rojos (saturación o clipping).
- **Modo Punto/Barra:** Algunos pueden configurarse para mostrar solo un punto que sube y baja o una barra sólida.

El Integrado LM3915

Para construir un vúmetro sin usar un microcontrolador, el componente estrella es el LM3915.

- **Función:** Es un controlador de visualización de puntos/barras de precisión que detecta niveles de voltaje y maneja 10 LEDs.
- **Logarítmico:** A diferencia del LM3914 (que es lineal para volímetros), el LM3915 tiene pasos de 3 dB, lo cual es perfecto para señales de audio.

Implementación en Microcontroladores

Si se usa un ATmega8535 para crear un vúmetro:

1. **Entrada:** La señal de audio entra por un canal del ADC.
2. **Acondicionamiento:** Es necesario un circuito de "offset" porque el audio tiene ciclos negativos y el ADC solo lee de 0 a 5V.
3. **Procesamiento:** El código calcula el valor promedio (o RMS) de las muestras recibidas.
4. **Salida:** Se encienden los LEDs o se dibuja una barra en una LCD 16x2.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 14, el circuito a armar es el siguiente:

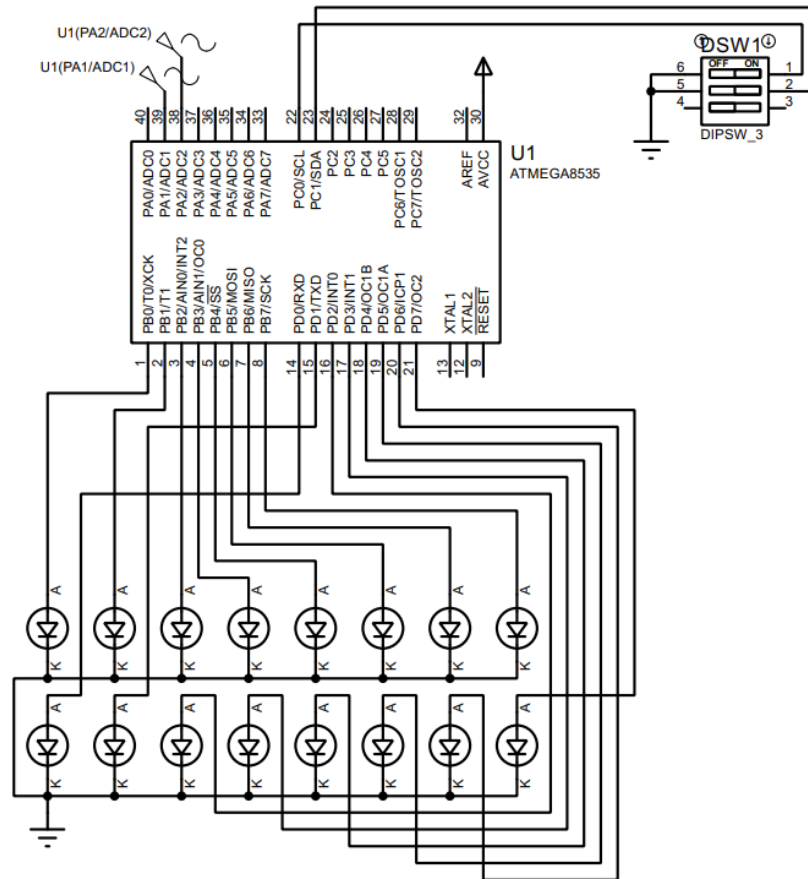


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen, el registro B y D se configuran como de salida (PB0-PB7) y (PD0-PD7) respectivamente, mientras que el registro A se configura como de entrada al igual que el registro C (PC0-PC1) activando las resistencias de pull-up.

Ports Settings

PORTA Port B Port C Port D

	Data Direction	Pullup/Output Value	
Bit 0	Out	0	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	Out	0	Bit 6
Bit 7	Out	0	Bit 7

Imagen 2. Configuraci3n del puerto B.

Ports Settings

PORTA Port B Port C Port D

	Data Direction	Pullup/Output Value	
Bit 0	In	P	Bit 0
Bit 1	In	P	Bit 1
Bit 2	In	T	Bit 2
Bit 3	In	T	Bit 3
Bit 4	In	T	Bit 4
Bit 5	In	T	Bit 5
Bit 6	In	T	Bit 6
Bit 7	In	T	Bit 7

Imagen 3. Configuraci3n del puerto C.

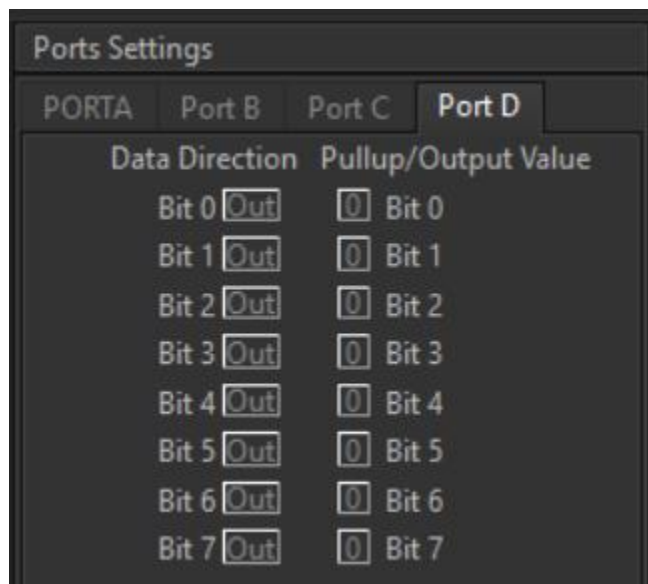


Imagen 4. Configuración del puerto D.

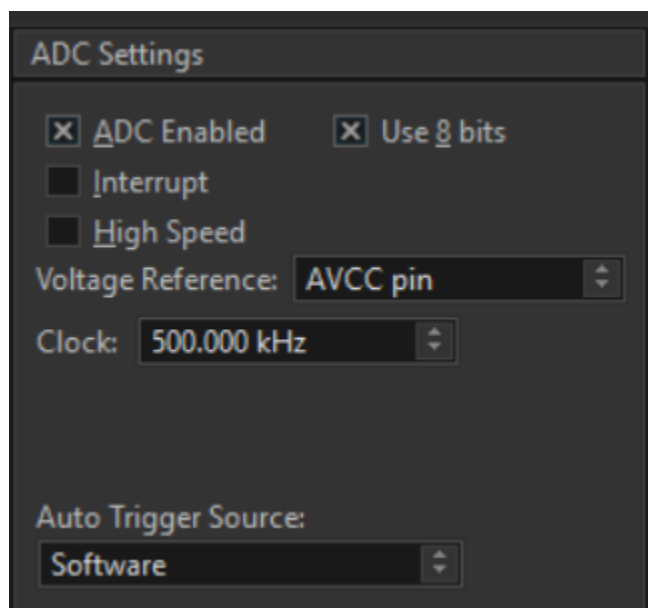


Imagen 5. Configuración del convertidor analógico digital.

Código del circuito de CodeVision:

```
1  /******  
2  This program was created by the CodeWizardAVR V4.06a  
3  Automatic Program Generator  
4  i   Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.  
5  http://www.hpinfotech.ro  
6  
7  Project : Pr  ctica 14 'V  metro'  
8  Version : 1.0  
9  Date    : 25/03/2026  
10 Author  : Garc  a Escamilla Bryan Alexis  
11 Company :  
12 Comments:  
13  
14  
15 Chip type      : ATmega8535  
16 Program type   : Application  
17 AVR Core Clock frequency: 1.000000 MHz  
18 Memory model   : Small  
19 External RAM size : 0  
20 Data Stack size : 128  
21 *****/  
22  
23 // I/O Registers definitions  
24 #include <mega8535.h>
```

```
26 // Delay functions  
27 #include <delay.h>  
28  
29 // Declare your global variables here  
30 unsigned char peak_hold[2] = {0, 0}; // Hold peak for each microphone  
31 unsigned char hold_counter[2] = {0, 0}; // Counter for hold duration  
32  
33 // ADC Voltage Reference: AVCC pin  
34 #define ADC_VREF_TYPE ((0<<REFS1) | (1<<REFS0) | (1<<ADLAR))  
35  
36 // Read the 8 most significant bits  
37 // of the AD conversion result  
38 // Read Voltage=read_adc*(Vref/256.0)  
39 unsigned char read_adc(unsigned char adc_input) {  
40     ADMUX=adc_input | ADC_VREF_TYPE;  
41     // Delay needed for the stabilization of the ADC input voltage  
42     delay_us(10);  
43     // Start the AD conversion  
44     ADCSRA|=(1<<ADSC);  
45     // Wait for the AD conversion to complete  
46     while ((ADCSRA & (1<<ADIF))==0);  
47     ADCSRA|=(1<<ADIF);  
48     return ADCH;  
49 }
```

```
51 // Function to convert ADC value to progressive LED pattern
52 unsigned char get_bar_pattern(unsigned char adc_value) {
53     // Define threshold levels for each LED
54     // Ajusta estos valores seg3n la sensibilidad del micr3fono
55     if (adc_value < 5) return 0x00; // Sin sonido (todos apagados)
56     else if (adc_value < 15) return 0x01; // LED 1 (PB0)
57     else if (adc_value < 30) return 0x03; // LED 1-2 (PB0-1)
58     else if (adc_value < 50) return 0x07; // LED 1-3 (PB0-2)
59     else if (adc_value < 70) return 0x0F; // LED 1-4 (PB0-3)
60     else if (adc_value < 90) return 0x1F; // LED 1-5 (PB0-4)
61     else if (adc_value < 120) return 0x3F; // LED 1-6 (PB0-5)
62     else if (adc_value < 160) return 0x7F; // LED 1-7 (PB0-6)
63     else return 0xFF; // Todos los LEDs (PB0-7)
64 }
```

```
66 // Function to convert ADC value to single LED position
67 unsigned char get_point_pattern(unsigned char adc_value) {
68     // Define threshold levels for each individual LED
69     if (adc_value < 10) return 0x00; // Sin sonido (todos apagados)
70     else if (adc_value < 25) return 0x01; // Solo LED 1 (PB0)
71     else if (adc_value < 40) return 0x02; // Solo LED 2 (PB1)
72     else if (adc_value < 55) return 0x04; // Solo LED 3 (PB2)
73     else if (adc_value < 70) return 0x08; // Solo LED 4 (PB3)
74     else if (adc_value < 85) return 0x10; // Solo LED 5 (PB4)
75     else if (adc_value < 100) return 0x20; // Solo LED 6 (PB5)
76     else if (adc_value < 115) return 0x40; // Solo LED 7 (PB6)
77     else return 0x80; // Solo LED 8 (PB7)
78 }
```



```

80 // Function to get hold pattern based on current value and peak hold
81 unsigned char get_hold_pattern(unsigned char adc_value, unsigned char mic_index) {
82     unsigned char bar = get_bar_pattern(adc_value); // barra normal
83     unsigned char peak = get_point_pattern(peak_hold[mic_index]); // pico como punto
84     return bar | peak; // mezcla barra + punto (HOLD visible)
85 }
86
87 // Function to update hold peak
88 void update_hold_peak(unsigned char adc_value, unsigned char mic_index) {
89     if (adc_value > peak_hold[mic_index]) {
90         peak_hold[mic_index] = adc_value;
91         hold_counter[mic_index] = 0;
92     }
93
94     if (++hold_counter[mic_index] >= 20) {
95         if (peak_hold[mic_index] >= 10)
96             peak_hold[mic_index] -= 10; // mÃ¡s rápido (ANTES 5)
97         else
98             peak_hold[mic_index] = 0;
99
100         hold_counter[mic_index] = 15;
101     }
102 }

```

```

104 unsigned char get_hold_mode(unsigned char adc_value, unsigned char mic_index) {
105     update_hold_peak(adc_value, mic_index);
106     return get_hold_pattern(adc_value, mic_index);
107 }
108
109 void main(void) {
110     // Declare your local variables here
111     unsigned char mic1_value, mic2_value;
112     unsigned char led_pattern1, led_pattern2;
113     unsigned char modo;
114     int temp1, temp2;
115     // FILTRO DE OFFSET (ruido base del micrÃfono)
116     // unsigned char base = 120; // AJUSTA este valor (100â€140 aprox)
117
118     // Input/Output Ports initialization
119     // Port A initialization
120     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
121     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
122     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
123     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

```

```

125 // Port B initialization
126 // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
127 DDRB=(1<<DDB7) | (1<<DDB6) | (1<<DDB5) | (1<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1) | (1<<DDB0);
128 // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
129 PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
130
131 // Port C initialization
132 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
133 DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
134 // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=P Bit0=P
135 PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (1<<PORTC1) | (1<<PORTC0);
136
137 // Port D initialization
138 // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
139 DDRD=(1<<DDD7) | (1<<DDD6) | (1<<DDD5) | (1<<DDD4) | (1<<DDD3) | (1<<DDD2) | (1<<DDD1) | (1<<DDD0);
140 // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
141 PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);
142
143 // Timer/Counter 0 initialization
144 // Clock source: System Clock
145 // Clock value: Timer 0 Stopped
146 // Mode: Normal top=0xFF
147 // OC0 output: Disconnected
148 TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
149 TCNT0=0x00;
150 OCR0=0x00;

```

```

152 // Timer/Counter 1 initialization
153 // Clock source: System Clock
154 // Clock value: Timer1 Stopped
155 // Mode: Normal top=0xFFFF
156 // OC1A output: Disconnected
157 // OC1B output: Disconnected
158 // Noise Canceler: Off
159 // Input Capture on Falling Edge
160 // Timer1 Overflow Interrupt: Off
161 // Input Capture Interrupt: Off
162 // Compare A Match Interrupt: Off
163 // Compare B Match Interrupt: Off
164 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
165 TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
166 TCNT1H=0x00;
167 TCNT1L=0x00;
168 ICR1H=0x00;
169 ICR1L=0x00;
170 OCR1AH=0x00;
171 OCR1AL=0x00;
172 OCR1BH=0x00;
173 OCR1BL=0x00;

```

```

175 // Timer/Counter 2 initialization
176 // Clock source: System Clock
177 // Clock value: Timer2 Stopped
178 // Mode: Normal top=0xFF
179 // OC2 output: Disconnected
180 ASSR=0<<AS2;
181 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
182 TCNT2=0x00;
183 OCR2=0x00;
184
185 // Timer(s)/Counter(s) Interrupt(s) initialization
186 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
187
188 // External Interrupt(s) initialization
189 // INT0: Off
190 // INT1: Off
191 // INT2: Off
192 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
193 MCUCSR=(0<<ISC2);
194
195 // USART initialization
196 // USART disabled
197 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

```

```

199 // Analog Comparator initialization
200 // Analog Comparator: Off
201 // The Analog Comparator's positive input is
202 // connected to the AIN0 pin
203 // The Analog Comparator's negative input is
204 // connected to the AIN1 pin
205 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
206
207 // ADC initialization
208 // ADC Clock frequency: 500.000 kHz
209 // ADC High Speed Mode: Off
210 // ADC Auto Trigger Source: Software
211 // Only the 8 most significant bits of
212 // the AD conversion result are used
213 ADMUX = ADC_VREF_TYPE;
214 ADCSRA=(1<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (1<<ADPS0);
215 SFIOR=(1<<ADHSM) | (0<<ADTS2) | (0<<ADTS1) | (0<<ADTS0);
216
217 // SPI initialization
218 // SPI disabled
219 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
220
221 // TWI initialization
222 // TWI disabled
223 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```

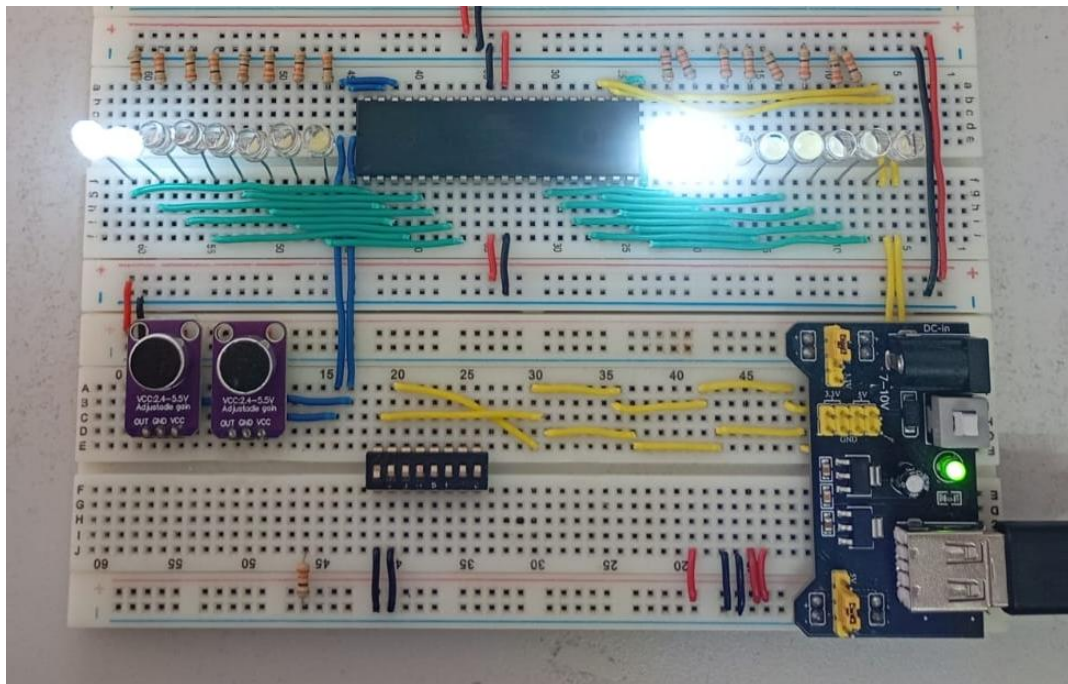
```

225 while (1) {
226     // Place your code here
227     mic1_value = read_adc(1);
228     mic2_value = read_adc(2);
229
230     // MIC 1
231     temp1 = mic1_value - 128;
232     if (temp1 < 0) temp1 = -temp1;
233     mic1_value = temp1;
234
235     // MIC 2
236     temp2 = mic2_value - 128;
237     if (temp2 < 0) temp2 = -temp2;
238     mic2_value = temp2;
239
240     // Leer modo
241     modo = PINC & 0x03;
242
243     switch (modo) {
244         case 0x00: // MODO BARRA
245             led_pattern1 = get_bar_pattern(mic1_value);
246             led_pattern2 = get_bar_pattern(mic2_value);
247             break;

```

```
248     case 0x01: // MODO PUNTO
249         led_pattern1 = get_point_pattern(mic1_value);
250         led_pattern2 = get_point_pattern(mic2_value);
251         break;
252     case 0x02: // MODO HOLD
253         led_pattern1 = get_hold_mode(mic1_value, 0);
254         led_pattern2 = get_hold_mode(mic2_value, 1);
255         break;
256     default: // SEGURIDAD
257         led_pattern1 = 0x00;
258         led_pattern2 = 0x00;
259         break;
260 }
261 delay_ms(30);
262
263 PORTB = led_pattern1;
264 PORTD = led_pattern2;
265 }
266 }
```

Circuito f3sico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Como en esta práctica se usa el ADC eso significa que es susceptible al ruido, por lo que se pueden dar lecturas incorrectas provenientes del MAX4466 Amplificador de Micrófono Electret, debido a esto se optó por agregar un filtro para despreciar el ruido. Esto se hizo porque en un principio más de la mitad de los leds permanecían encendidos a pesar de que el ruido alrededor fuera muy bajo, sin embargo, cuando se quiere medir la intensidad del audio de una canción se tiene que acercar la salida de audio muy cerca de los micrófonos para que se observara el cambio de intensidad de esta.

Se modificó el código para que el HOLD bajara más rápido y este se pudiera distinguir del modo BARRA en cambios muy rápidos de la intensidad del audio.

- **Meléndez Macedonio Rodrigo**

Para esta práctica se desarrolló un vúmetro completamente funcional, y para ello tomamos ciertas consideraciones al momento de configurarlo, pues por ejemplo tuvimos que considerar que el ADC es muy sensible al ruido, lo que ocasionaba que las lecturas de los audios no fueran muy precisas por lo que decidimos agregar un filtro para eliminar el ruido, de esta forma obtuvimos un vúmetro más preciso, sin embargo, hubo un pequeño problema con esta implementación ya que ahora para medir la intensidad de audio de una canción tenemos que acercar la salida del audio muy pegada a los micrófonos para obtener buenos resultados por parte los LEDs. A pesar de ese pequeño detalle, el vúmetro funciona correctamente y cumple su función para cada uno de los modos diseñados.

BIBLIOGRAFÍA

- (s.f.). Vúmetro. Wikipedia. Recuperado el 13 de abril de 2026, de <https://es.wikipedia.org/wiki/V%C3%BAmetro>
- (s.f.). VU Meter Circuits. All About Circuits. Recuperado el 13 de abril de 2026, de <https://www.allaboutcircuits.com/>
- Elliot, R. (2000). VU and PPM Audio Metering. Sound-au. Recuperado el 13 de abril de 2026, de <https://www.sound-au.com/project55.htm>